

Dendê Eventos – Gerenciador de Eventos em Kotlin

Autor: Bruno Sampaio Silva
Levi Falcão de Queiroz
Moisés de Souza Oliveira
Raphael dos Santos Moreira

Agenda

Apresentar o sistema Dendê
Eventos, suas funcionalidades e
arquitetura.

1. Visão Geral:

O projeto e seus objetivos

2. Arquitetura:

Estrutura e organização do
código

3. User Stories:

Usuários, Eventos e
Ingressos

4. US9 e US10:

Ativar e Desativar Evento

5. Considerações:

Conclusões e próximos
passos

Introdução



- Dendê-Software House
- Somos Trainees (Dendezeiros)
- Software de Gestão de Eventos

Anteriormente



- Software de Gerenciamento de Eventos via terminal
- 15 user stories
- Implementação sem funções
- Dentro de um único Arquivo
- Hoje vamos apresentar o código refatorado

Visão Geral do Projeto

- Produto Principal – Dendê Eventos
 - Aplicativo voltado à gestão, divulgação e organização de eventos culturais, acadêmicos e sociais, promovendo a conexão entre organizadores e público de forma simples, eficiente e acessível

Dendê Softhouse

Empresa baiana de desenvolvimento de software, sediada em Salvador – BA.

Especializada na criação de soluções digitais com foco em impacto local e inovação tecnológica.

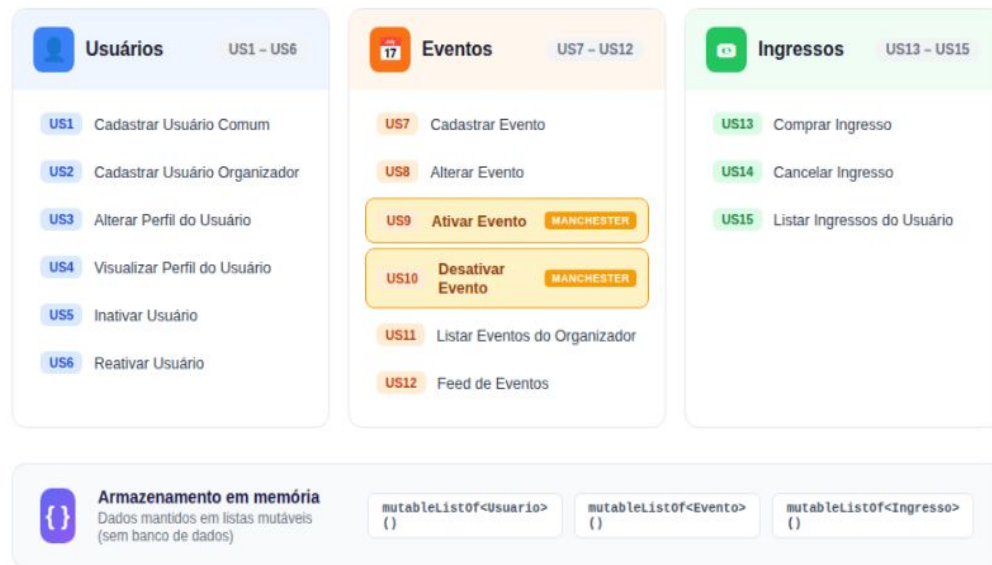
User Stories



- Implementa 15 User Stories em domínios: Usuários (US1-US6), Eventos (US7-US12) e Ingressos (US13-US15). Utiliza armazenamento em memória com listas mutáveis.

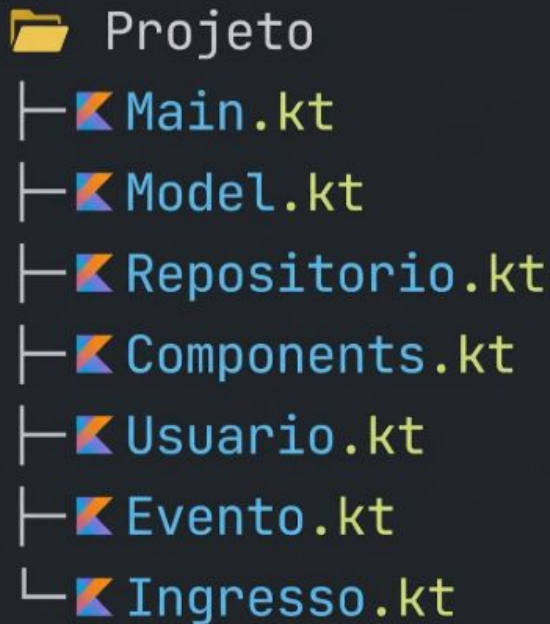
User Stories — Dendê Eventos

15 User Stories em 3 domínios • Armazenamento em memória com listas mutáveis



Arquitetura do Código

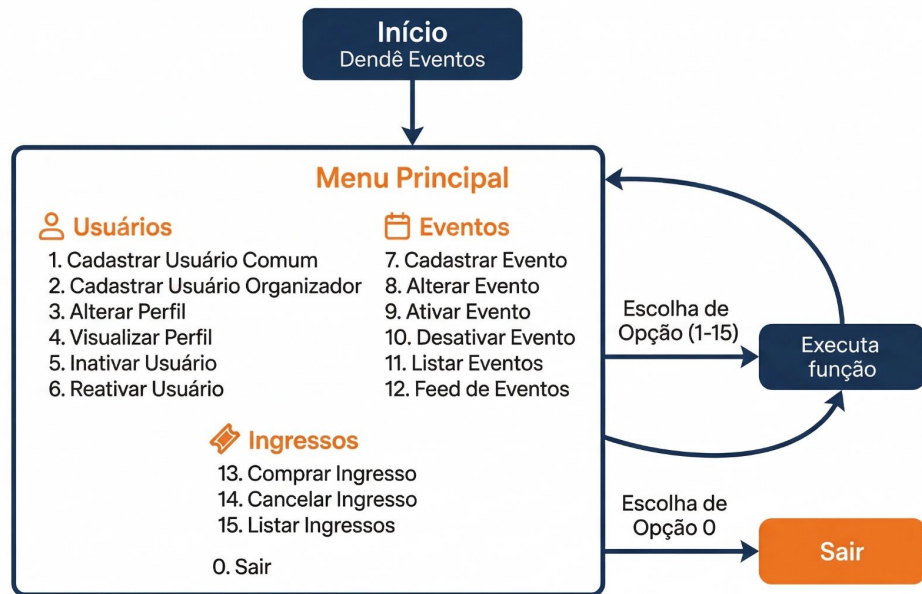
- Estrutura de arquivos após refatoração:
 - Organização dos Arquivos
 - Códigos Semelhantes ou relacionados, dentro de um único [arquivo.kt](#)



```
Projeto
├─ Main.kt
├─ Model.kt
├─ Repositorio.kt
├─ Components.kt
├─ Usuario.kt
├─ Evento.kt
└─ Ingresso.kt
```

Arquitetura do Código

- Organização do arquivo Main.kt:
 - Utilização do When para decisão do usuário
 - Execução da função de acordo com a decisão do usuário



Arquitetura do Código



- Organização do arquivo Model.kt:
 - Define as estruturas de dados do sistema
 - Contém os data classes e enums utilizados em todo o projeto

```
Model.kt — Data classes e Enums

// ---- Enums ----

enum class Sexo {
    MASCULINO, FEMININO, OUTRO, PREFIRO_NAO_DIZER
}

enum class TipoEvento {
    SOCIAL, CORPORATIVO, ACADEMICO, CULTURAL_ENTRETENIMENTO,
    RELIGIOSO, ESPORTIVO, FEIRA, CONGRESSO, OFICINA, ...
}

enum class ModalidadeEvento { PRESENCIAL, REMOTO, HIBRIDO }
enum class StatusIngresso { ATIVO, CANCELADO }

// ---- Data Classes ----

data class Empresa(var cnpj: String, var razaoSocial: String, var nomeFantasia: String)

data class Usuario(
    val id: String = UUID.randomUUID().toString(),
    var nome: String, var dataNascimento: LocalDate,
    var sexo: Sexo, val email: String,
    var senha: String, var ativo: Boolean = true,
    var empresa: Empresa? = null
)

data class Evento(
    val id: String = UUID.randomUUID().toString(),
    val organizadorId: String, var nome: String,
    var inicio: LocalDateTime, var fim: LocalDateTime,
    var tipo: TipoEvento, var modalidade: ModalidadeEvento,
    var capacidade: Int, var ativo: Boolean,
    var preco: Double, var estornaEmCancelamento: Boolean, ...
)

data class Ingresso(
    val id: String = UUID.randomUUID().toString(),
    val usuarioId: String, val eventId: String,
    var status: StatusIngresso = StatusIngresso.ATIVO,
    var valorPago: Double
)
```

Arquitetura do Código

- Organização do arquivo

Repositorio.kt:

- Simula um banco de dados em memória
- Centraliza todas as operações de salvar, alterar, excluir e buscar dados

```
Repositorio.kt — CRUD em memória

// Listas que funcionam como banco de dados em memória
val usuarios = mutableListOf<Usuario>()
val eventos = mutableListOf<Evento>()
val ingressos = mutableListOf<Ingresso>()

// ---- Funções de Usuario ----
fun salvarUsuario(usuario: Usuario) { usuarios.add(usuario) }
fun alterarUsuario(u: Usuario) { val i = usuarios.indexOfFirst { it.id == u.id }; if (i >= 0) usu
fun excluirUsuario(id: String) { usuarios.removeIf { it.id == id } }
fun findUsuarioPorEmail(email: String): Usuario? = usuarios.find { it.email.equals(email, ignoreCase = true) }
fun filterUsuariosAtivos(): List<Usuario> = usuarios.filter { it.ativo }

// ---- Funções de Evento ----
fun salvarEvento(evento: Evento) { eventos.add(evento) }
fun alterarEvento(e: Evento) { val i = eventos.indexOfFirst { it.id == e.id }; if (i >= 0) even
fun excluirEvento(id: String) { eventos.removeIf { it.id == id } }
fun findEventoPorId(id: String): Evento? = eventos.find { it.id == id }
fun filterEventosPorOrganizador(orgId: String) = eventos.filter { it.organizadorId == orgId }
fun filterEventosAtivosDisponiveis() = eventos.filter { it.ativo && it.fim.isAfter(now()) && ... }

// ---- Funções de Ingresso ----
fun salvarIngresso(ing: Ingresso) { ingressos.add(ing) }
fun alterarIngresso(i: Ingresso) { val idx = ingressos.indexOfFirst { it.id == i.id }; if (idx >= 0
fun excluirIngresso(id: String) { ingressos.removeIf { it.id == id } }
fun findIngressoPorId(id: String): Ingresso? = ingressos.find { it.id == id }
fun filterIngressosPorUsuario(uid: String) = ingressos.filter { it.usuarioId == uid }
fun filterIngressosAtivosPorEvento(eid: String) = ingressos.filter { it.eventoId == eid && it.status == StatusIn
fun contarIngressosAtivos(eid: String): Int = ingressos.count { it.eventoId == eid && it.status == StatusIn
```

Arquitetura do Código



- Organização do arquivo components.kt
 - Centraliza as funções de entrada e saída de dados
 - Garante que os dados lidos do usuário sejam sempre válidos antes de prosseguir

```
Components.kt — Funções de entrada/saída + Formulários

// ---- 4 Funções de leitura e escrita ----

fun readInt(message: String, errorMessage: String, range: IntRange = 0..Int.MAX_VALUE): Int {
    while (true) {
        print(message)
        val valor = readLine()?.trim()?.toIntOrNull()
        if (valor != null && valor in range) return valor
        println(errorMessage)
    }
}

fun readDouble(message: String, errorMessage: String,
    minValue: Double = 0.0, maxValue: Double = Double.MAX_VALUE): Double

fun readString(message: String, errorMessage: String, minLength: Int = 0): String

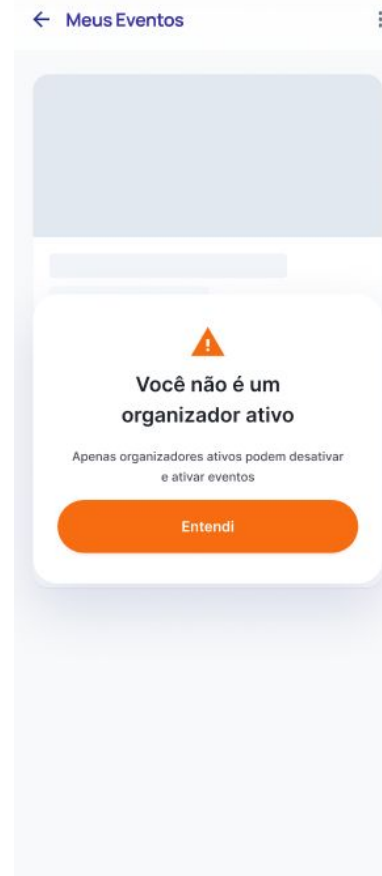
fun printTable(header: String, items: List<Any>)

// =====
// Formulário 1: Cadastrar Usuário (usando Components)
// =====
fun cadastrarUsuarioComum() {
    val nome = readString("Nome: ", "O nome não pode ser vazio.", minLength = 1)
    val sexo = lerSexo() // usa readInt internamente
    val email = readString("E-mail: ", "O e-mail não pode ser vazio.", minLength = 1)
    val senha = readString("Senha: ", "A senha não pode ser vazia.", minLength = 1)
    ...
}

// =====
// Formulário 2: Cadastrar Evento (usando Components)
// =====
fun cadastrarEvento() {
    val emailOrg = readString("E-mail do organizador: ", "...", minLength = 1)
    val nome = readString("Nome do evento: ", "...", minLength = 1)
    val cap = readInt("Capacidade: ", "Deve ser > 0.", range = 1..Int.MAX_VALUE)
    val preco = readDouble("Preço: ", "Valor inválido.")
    val tipo = lerTipoEvento() // usa readInt + printTable
    val modal = lerModalidade() // usa readInt
    ...
    printTable("EVENTOS", listaEventos) // exibição tabular
}
```

Ativar Evento

- Regras de negócio:
 - Valida organizador ativo e se permite ele ativar



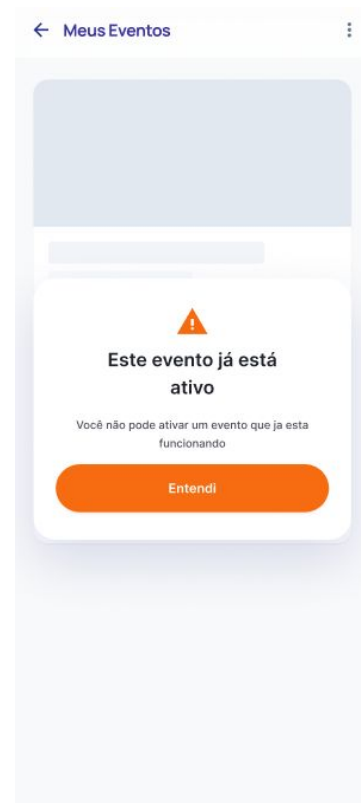
Ativar Evento

- Regras de negócio:
 - Impede ativação de evento já encerrado



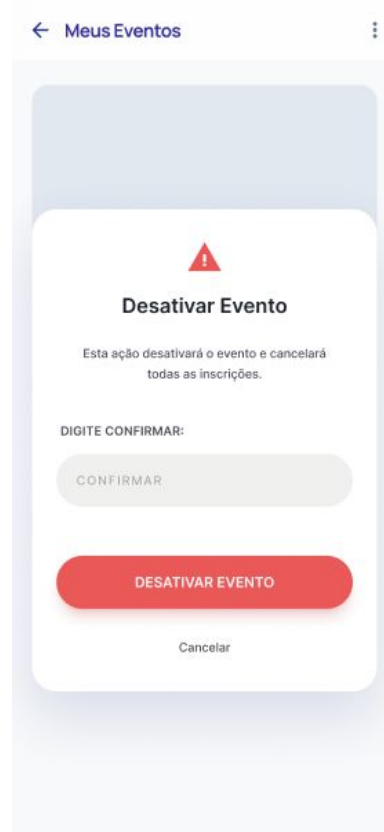
Ativar Evento

- Regras de negócio:
 - Informa se evento já está ativo e não permite ativá-lo novamente



Desativar Evento

- Regras de negócio:
 - Desativa evento e suspende vendas



Desativar Evento

- Regras de negócio:
 - Cancela todos os ingressos ativos



Considerações finais



- **Refatoração** — separação em arquivos por responsabilidade com chamadas entre Main, domínios, Repositorio e Components
- **Ativar Evento** — validação do organizador, status do evento e impedimento de ativar eventos encerrados
- **Desativar Evento** — cancelamento automático de ingressos ativos e cálculo de reembolso com taxa de estorno

Referências



- Kotlin Documentation – kotlinlang.org.
- JetBrains IntelliJ IDEA.
- Material da disciplina Desenvolvimento Mobile – UNEX.